

Exclusion criteria

August 19, 2026

Analyses for Mosteiro and Blasi and Mosteiro et al were done on all available translations. This led to outliers. Hedwig removed outliers by cutting D_order and D_structure on sigmas from the mean; that is methodologically not very justifiable. We also in several parts selected books with >80% and >90% of all available verses. That seems like a better cut.

```
[1]: import sys
from pathlib import Path
import os
import pandas as pd

# Find project root (adjust if needed)
project_root = Path.cwd().parents[1]
sys.path.append(str(project_root / "scripts" / "NounNounCompounds"))

from final_paper_plots import get_verse_len_df

[2]: from wordorderbibles.util import BOOK_ID_NAME
from wordorderbibles import data
import matplotlib.pyplot as plt
import numpy as np

[3]: BIBLES_DIR = os.path.join(Path.cwd().parents[2], 'paralleltext', 'bibles',
    ↪ 'corpus')

[4]: book_id_name = {int(k): v for k, v in BOOK_ID_NAME.items()}
book_id_name = pd.DataFrame(list(book_id_name.items()), columns=["book_id",
    ↪ "book"])

[5]: get_verse_len_df('eng', [], book_id_name, BIBLES_DIR)

[5]:
```

	file	book	n_verses	max_verses
0	eng-x-bible-world.txt	Matthew	1071	1071
1	eng-x-bible-world.txt	Mark	678	678
2	eng-x-bible-world.txt	Luke	1151	1151
3	eng-x-bible-world.txt	John	879	879
4	eng-x-bible-world.txt	Acts	1007	1007
..	...	...	...	...
205	eng-x-bible-amplified.txt	Mark	676	678

206	eng-x-bible-amplified.txt	Luke	1151	1151
207	eng-x-bible-amplified.txt	John	879	879
208	eng-x-bible-amplified.txt	Acts	1007	1007
209	eng-x-bible-amplified.txt	Revelation	404	404

[210 rows x 4 columns]

This computes the `max_verses` for a given language. However, the PBC is a **parallel** corpus. That means that the maximum number of verses is a universal, i.e., the same for all languages. So, we need to rewrite this function in a language-agnostic way.

```
[6]: file_dfs = []
for file in os.listdir(BIBLES_DIR):
    bible = data.parse_file(os.path.join(BIBLES_DIR, file), 'pbc').
    ↪join_by_toc()[2]
    book_id_n_verses = {k: len(v) for k, v in bible.items()}
    file_df = pd.DataFrame(list(book_id_n_verses.items()), columns=['book_id',
    ↪'n_verses'])
    file_df['file'] = file
    file_dfs.append(file_df)
df = pd.concat(file_dfs)
```

```
[7]: id_max = df[['n_verses', 'book_id']].groupby('book_id').max().reset_index().
    ↪rename(columns={'n_verses': 'max_verses'})
```

```
[8]: full_df = df.merge(id_max, on='book_id', how='left')
```

```
[9]: assert len(full_df[full_df['max_verses'] < full_df['n_verses']]) == 0
```

```
[10]: full_df.head()
```

```
[10]:
```

	book_id	n_verses	file	max_verses
0	1	1533	rus-x-bible-newworld.txt	1534
1	2	1213	rus-x-bible-newworld.txt	1214
2	3	859	rus-x-bible-newworld.txt	860
3	4	1288	rus-x-bible-newworld.txt	1288
4	5	959	rus-x-bible-newworld.txt	960

Now I have a dataframe called `df` with four columns:

- `book_id`: an identifier for a book of the bible
- `n_verses`: the number of verses contained in that book for that translation
- `file`: the file name for that translation
- `max_verses`: the maximum number of verses for that book, for any translation.

I want a program that gives runs three analyses, at the levels of:

1. each of six specific books (the keys of the dictionary `book_id_name`)
2. the same six specific books combined
3. all books of the bible (all possible `book_id`)

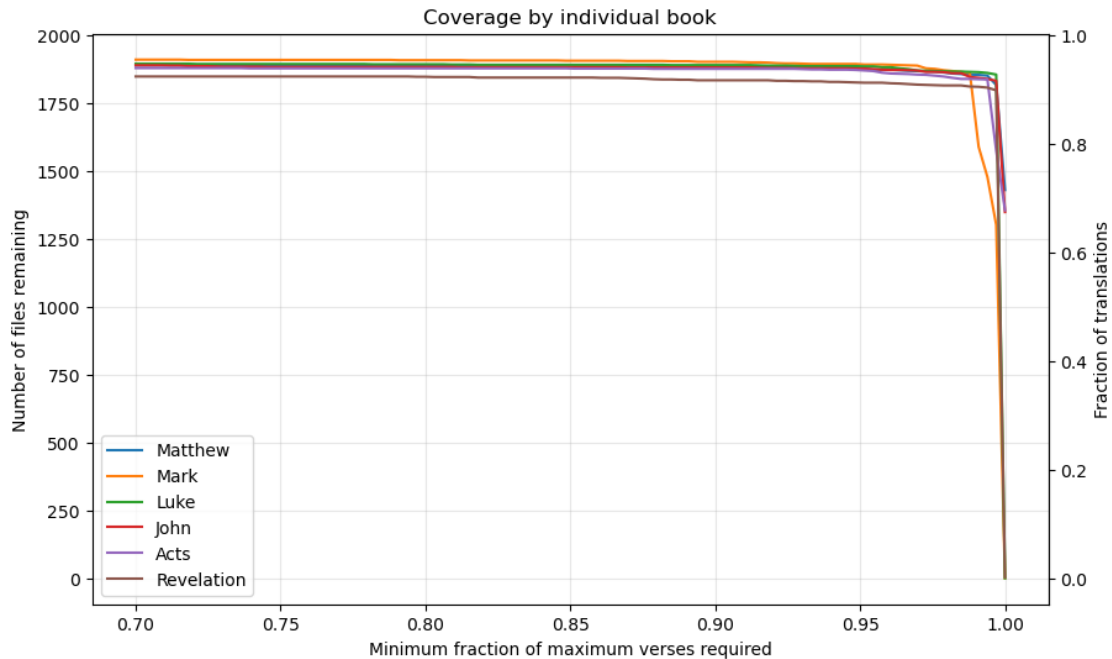
For each of these three, I want a graph. The x axis is the minimum numbers of verses required, expressed as a fraction of max_verses. The y axis is how many files remain after requiring that minimum fraction of verses.

```
[11]: total_files = full_df['file'].nunique()

# 1. Each of the six books independently
plt.figure(figsize=(10, 6))
for book_id, book in book_id_name[['book_id', 'book']].values:
    book_df = full_df[full_df['book_id'] == book_id].reset_index()
    assert book_df['file'].nunique() == len(book_df)
    assert book_df['max_verses'].nunique() == 1
    n_files = len(book_df)
    fractions, counts = [], []
    for fraction in np.linspace(0.7, 1, 100):
        n_valid = len(book_df[book_df['n_verses'] >= fraction *
↪book_df['max_verses']])
        valid_fraction = n_valid
        fractions.append(fraction)
        counts.append(valid_fraction)
    plt.plot(fractions, counts, label=book)
plt.xlabel("Minimum fraction of maximum verses required")
plt.ylabel("Number of files remaining")
plt.title("Coverage by individual book")
plt.legend()
plt.grid(alpha=0.3)
ax = plt.gca()

secax = ax.secondary_yaxis(
    "right",
    functions=(
        lambda y: y / total_files,
        lambda y: y * total_files,
    ),
)

secax.set_ylabel("Fraction of translations")
plt.show()
```



```
[12]: def coverage_counts(df, book_ids, min_fraction):
    fractions = np.linspace(min_fraction, 1, 200)
    counts = []

    subset = df[df["book_id"].isin(book_ids)].reset_index()

    for fraction in fractions:
        subset['is_valid'] = subset['n_verses'] >= fraction * \
        subset['max_verses']
        counts.append(0)
        # File must pass the threshold for every selected book
        for file, grp in subset.groupby('file'):
            if len(grp) < len(book_ids):
                continue
            if all(grp['is_valid'].tolist()):
                counts[-1] += 1

    return fractions, counts

def plot_coverage(df, book_ids, title, min_fraction):
    fractions, counts = coverage_counts(df, book_ids, min_fraction)
    plt.plot(fractions, counts, label=title)
```

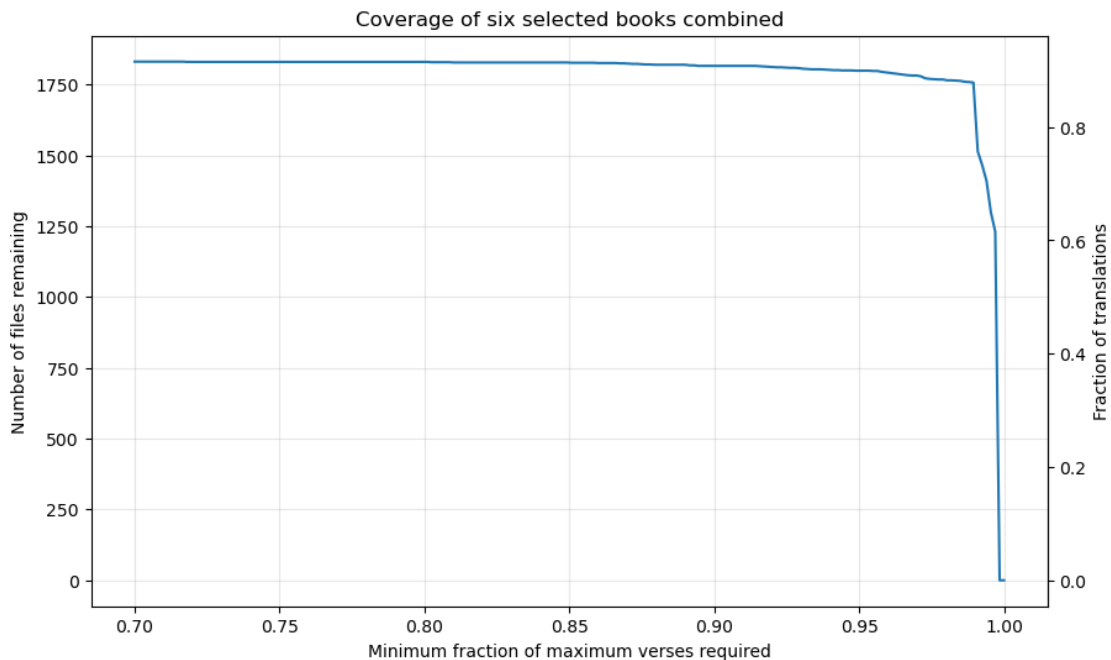
```
[13]: # 2. Six books combined
plt.figure(figsize=(10, 6))

plot_coverage(
    full_df,
    book_id_name['book_id'].tolist(),
    "Six books combined",
    0.7
)

plt.xlabel("Minimum fraction of maximum verses required")
plt.ylabel("Number of files remaining")
plt.title("Coverage of six selected books combined")
plt.grid(alpha=0.3)
ax = plt.gca()

secax = ax.secondary_yaxis(
    "right",
    functions=(
        lambda y: y / total_files,
        lambda y: y * total_files,
    ),
)

secax.set_ylabel("Fraction of translations")
plt.show()
```



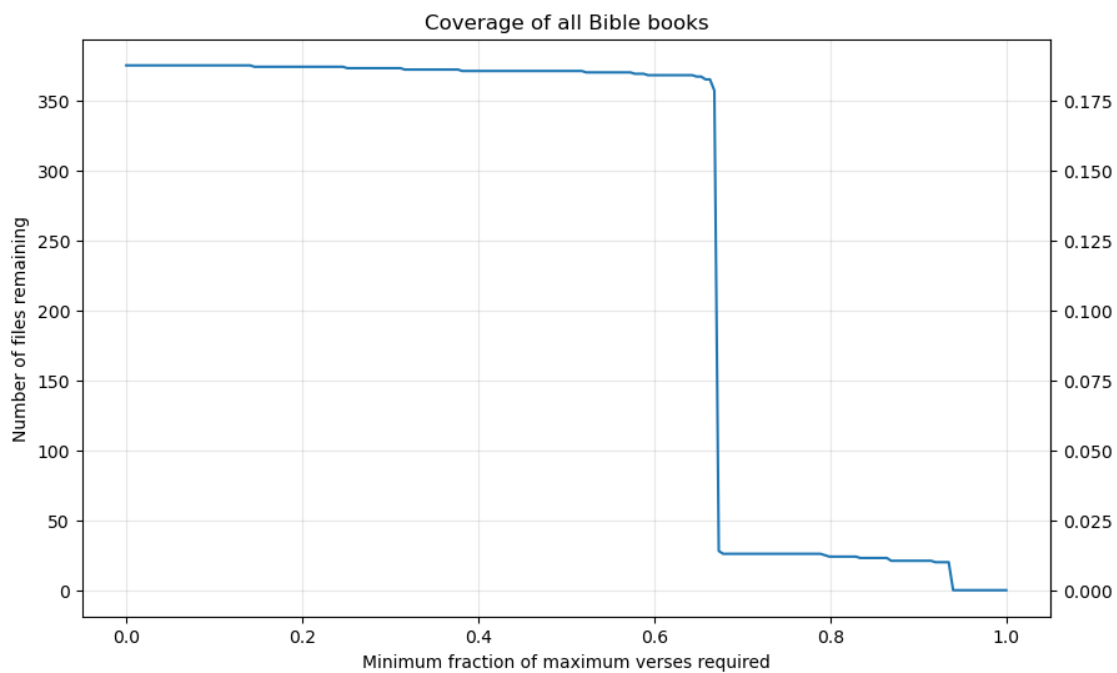
```
[14]: # 3. All books combined except the apocryphal ones
plt.figure(figsize=(10, 6))

plot_coverage(
    full_df,
    [el for el in full_df["book_id"].unique() if el <= 66],
    "All books",
    0.0
)

plt.xlabel("Minimum fraction of maximum verses required")
plt.ylabel("Number of files remaining")
plt.title("Coverage of all Bible books")
plt.grid(alpha=0.3)
ax = plt.gca()

secax = ax.secondary_yaxis(
    "right",
    functions=(
        lambda y: y / total_files,
        lambda y: y * total_files,
    ),
)

plt.show()
```

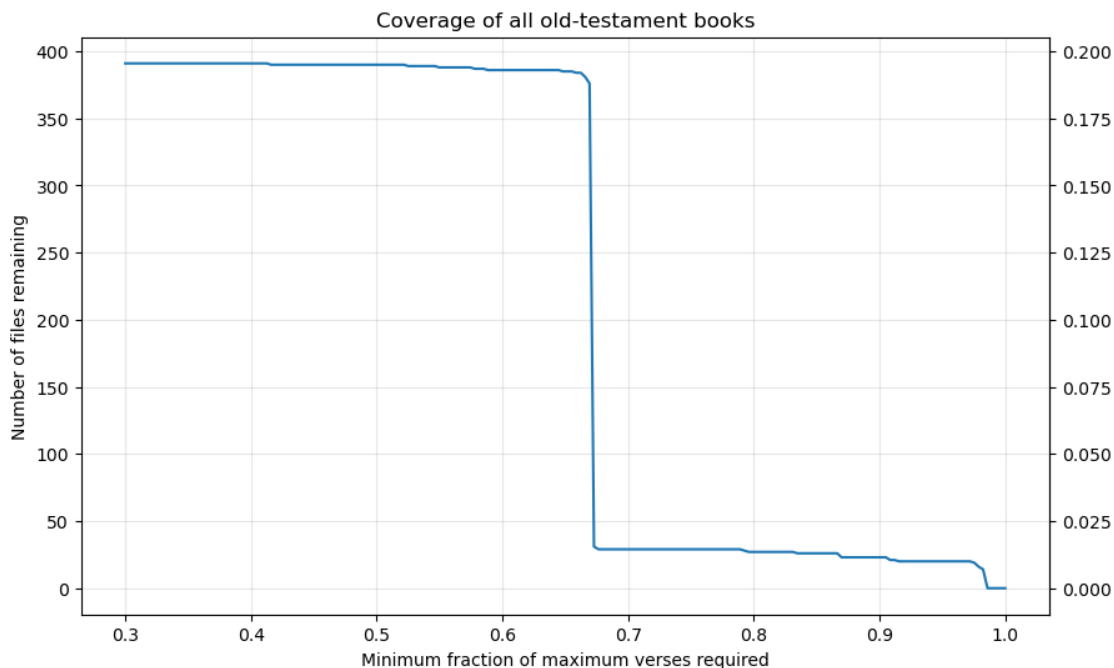


The shoulder is very likely due to the old and/or new testaments not being available for one or the other. Let's break up the old and new testaments.

```
[15]: # 4. Old testament
plt.figure(figsize=(10,6))
plot_coverage(full_df, [el for el in full_df['book_id'].unique() if el <=39],
               'Old testament', 0.3)
plt.xlabel("Minimum fraction of maximum verses required")
plt.ylabel("Number of files remaining")
plt.title("Coverage of all old-testament books")
plt.grid(alpha=0.3)
ax = plt.gca()

secax = ax.secondary_yaxis(
    "right",
    functions=(
        lambda y: y / total_files,
        lambda y: y * total_files,
    ),
)

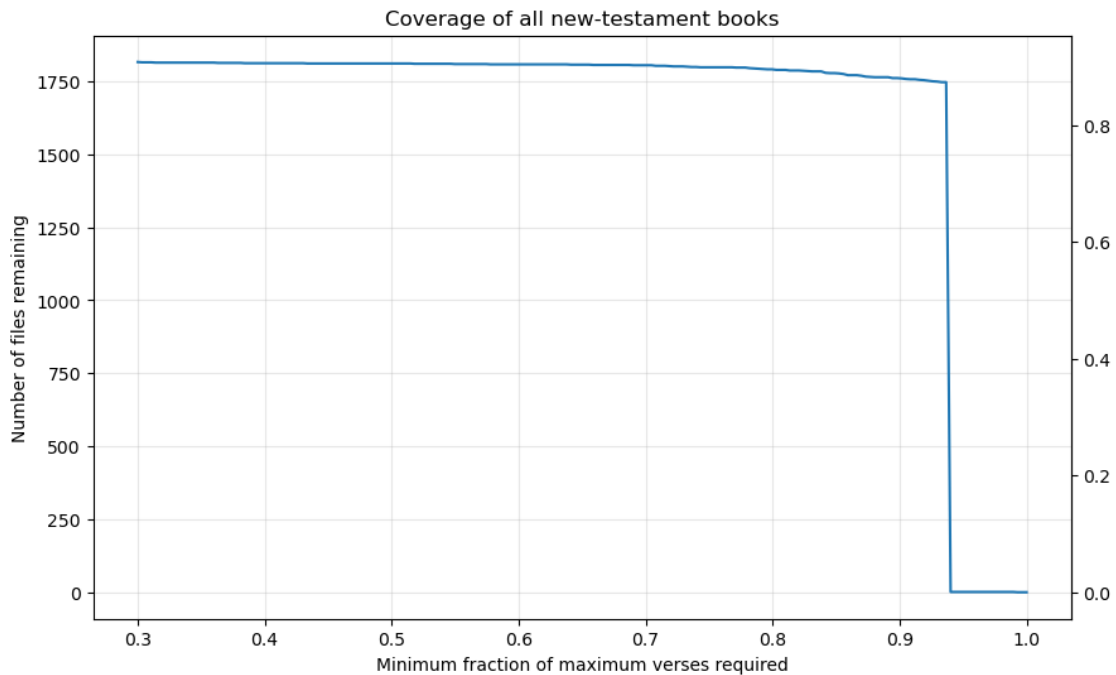
plt.show()
```



```
[16]: # 5. New testament
plt.figure(figsize=(10,6))
plot_coverage(full_df, [el for el in full_df['book_id'].unique() if el >=40 and
    el <= 66], 'New testament', 0.3)
plt.xlabel("Minimum fraction of maximum verses required")
plt.ylabel("Number of files remaining")
plt.title("Coverage of all new-testament books")
plt.grid(alpha=0.3)
ax = plt.gca()

secax = ax.secondary_yaxis(
    "right",
    functions=(
        lambda y: y / total_files,
        lambda y: y * total_files,
    ),
)

plt.show()
```



1 Conclusions

- Do not use old testament because there are too few bibles
- We can do analysis at the full-new-testament, 6-books-combined, or individual-book-out-of-

the-6 level

- For all of those, it is safe to require that at 90% of all possible verses for each book are available before including the translation in the analysis

[]: